```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import f1_score
from sklearn.metrics import roc_auc_score, accuracy_scor
from xgboost import XGBClassifier # importing the wantec
```

```
from google.colab import files
upload = files.upload()
```

Choose Files 2 files

dataset_C_training.csv(text/csv) - 878261 bytes, last modified: n/a - 100% done
dataset_C_testing.csv(text/csv) - 864158 bytes, last modified: n/a - 100% done

```
Saving dataset_C_training.csv to dataset_C_training (1).csv
Saving dataset_C_testing.csv to dataset_C_testing (1).csv
```

```
train = pd.read_csv('dataset_C_training.csv')
test = pd.read_csv('dataset_C_testing.csv') #reading the          ts
```

```
train.head(10)
```

	respondent_id	covid_concern	covid_knowledge	behavioral_antivira
0	1	3.0	1.0	
1	2	2.0	1.0	
2	3	0.0	2.0	
3	4	2.0	2.0	
4	5	1.0	1.0	
5	6	3.0	2.0	
6	7	3.0	1.0	
7	8	3.0	1.0	
8	9	2.0	2.0	
9	10	2.0	1.0	

10 rows × 31 columns

```
test.head(10)
```

	respondent_id	covid_concern	covid_knowledge	behavioral_antivira
0	4757	1.0	1.0	
1	4758	0.0	0.0	
2	4759	1.0	1.0	
3	4760	3.0	1.0	
4	4761	1.0	1.0	
5	4762	2.0	2.0	
6	4763	1.0	1.0	
7	4764	3.0	1.0	
8	4765	3.0	2.0	
9	4766	1.0	2.0	

10 rows × 30 columns

```
test_ids = test["respondent_id"]
```

```
TARGET = "covid_vaccine"
```

```
x = train.drop(columns=[TARGET])
y = train[TARGET] #Features and target
```

```
categorical_cols = x.select_dtypes(include=["object"]).columns.toli
numerical_cols = [
    col for col in x.columns
    if col not in categorical_cols] #identify columns types
```

```
numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median"))
    ]
)
categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]
)
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numerical_cols),
        ("cat", categorical_transformer, categorical_cols)
    ]
)
 #preprocessing
```

```
X_train, X_valid, y_train, y_valid = train_test_split(x,y,test_size
```

```
model = XGBClassifier(
    n_estimators=1000,
    max_depth=5,
    learning_rate=0.03,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42
)
pipeline = Pipeline(
    steps=[("preprocessor", preprocessor),("model", model)]) #model
```

```python
pipeline.fit(X_train,y_train) #train
```

Pipeline

preprocessor: ColumnTransformer

num	**cat**
SimpleImputer	SimpleImputer
	OneHotEncoder

XGBClassifier

```python
valid_probs = pipeline.predict_proba(X_valid)[:, 1]
valid_preds = (valid_probs > 0.5).astype(int)
print("Accuracy:", accuracy_score(y_valid, valid_preds))
print("ROC-AUC:", roc_auc_score(y_valid, valid_probs))
print("F1-Score:", f1_score(y_valid, valid_preds))
```

```
Accuracy: 0.7846638655462185
ROC-AUC: 0.8128025442561111
F1-Score: 0.6434782608695652
```

```python
test_probs = pipeline.predict_proba(test)[:, 1]
submission = pd.DataFrame({
    "respondent_id": test_ids,
    "covid_vaccine": test_probs
})
submission.to_csv("submission.csv", index=False)
print("submission.csv saved successfully!")
```

```
submission.csv saved successfully!
```